

Unity IL2CPP Metadata Deep Dive

Understanding the Magic Bytes • Encryption • Runtime Decryption • Dumping Techniques

1. What is Unity IL2CPP?

IL2CPP (Intermediate Language To C++) is Unity's high-performance scripting backend. Instead of running C# code through the Mono JIT interpreter, IL2CPP converts your C# scripts into native C++ code, which is then compiled into a platform-specific binary (**libil2cpp.so** on Android, **GameAssembly.dll** on Windows).

This conversion removes all original type information (class names, method signatures, field offsets, string literals) from the binary. To keep the managed runtime working, Unity stores that information in a separate binary file called **global-metadata.dat**.

2. The Metadata Magic Header

Every legitimate (decrypted) **global-metadata.dat** file begins with a fixed signature followed by a version number:

Bytes	Meaning	Value
AF 1B B1 FA	Magic / Sanity Signature	0xFAB11BAF (little-endian)
1F 00 00 00	Metadata Format Version	31 (common in Unity 2021–2022)

Full 8-byte pattern commonly searched in memory:

AF 1B B1 FA 1F 00 00 00 → Valid decrypted metadata, version 31

Common Metadata Versions

Version	Hex (LE)	Typical Unity Range
24	18 00 00 00	Unity 2018–2019
27	1B 00 00 00	Unity 2020–2021
29	1D 00 00 00	Unity 2021–2022
31	1F 00 00 00	Unity 2021.3 / 2022.x
35+	23 00 00 00+	Unity 6 (new formats)

3. Why Games Encrypt Metadata

The **global-metadata.dat** file contains the complete blueprint of the game's managed code. Anyone with a valid copy can use tools (IL2CppDumper, IL2CplInspector, Cpp2IL, Rodroid, etc.) to recover almost every class, method, field offset and string. This makes reverse-engineering, modding, and cheating significantly easier.

To protect against this, many commercial games encrypt, obfuscate, rename, or embed the metadata file. On disk the header no longer starts with **AF 1B B1 FA**.

4. Why the Memory Search Still Works

Encryption only protects the **on-disk** file. When the game runs, Unity's own IL2CPP runtime **must** decrypt the metadata into the exact format the engine expects. The runtime contains a hard-coded check:

```
s_GlobalMetadataHeader = (const Il2CppGlobalMetadataHeader*)s_GlobalMetadata;  
IL2CPP_ASSERT(s_GlobalMetadataHeader->sanity == 0xFAB11BAF); // AF 1B B1 FA  
IL2CPP_ASSERT(s_GlobalMetadataHeader->version == 31);
```

If the decrypted data does not start with the correct magic, the game itself will crash. Therefore, after the game has fully loaded, a clean copy of the metadata (starting with AF 1B B1 FA ...) almost always exists somewhere in process memory.

Key Insight: The search works on the majority of Unity IL2CPP games not because of weak encryption, but because the Unity engine itself forces the data back into the standard format at runtime.

5. Practical Dumping Workflow (Game Guardian)

1. Fully launch the target game and reach a stable state (main menu or in-game).
2. Attach Game Guardian and select the game process.
3. Set search ranges to **All** (or at minimum CA + Anonymous + Other).
4. Perform a Hex search for the full pattern: h AF 1B B1 FA 1F 00 00 00 (or just the 4-byte magic if the longer pattern fails).
5. When a hit is found, navigate to that address and dump a generous region (typically 2–8 MB or larger than the original encrypted file size).
6. Open the dumped file in a hex editor and verify the first 8 bytes are exactly the expected magic + version.
7. Feed the clean metadata + the corresponding libil2cpp.so into a dumper (Rodroid, Il2CppDumper, Il2CppInspector, etc.).

6. When the Simple Search Fails

A minority of heavily protected titles never leave a clean header in memory, wipe the magic after loading, use multi-stage decryption, split the metadata, or ship with completely custom formats (including Unity 6's new sanity constant 0xC1C21EF0). In those cases pure header search is insufficient and more advanced techniques are required:

- Frida scripts that locate the decrypted buffer via known function offsets or pattern scanning
- Zygisk / Magisk modules that dump metadata + libil2cpp at runtime
- Full process memory dumps followed by offline searching
- Reverse-engineering the decryption routine inside libil2cpp.so

7. Complete Summary

Concept	Explanation
AF 1B B1 FA	Fixed magic number (0xFAB11BAF) that identifies a real IL2CPP global-metadata file
1F 00 00 00	Metadata version 31 (little-endian) — common for Unity 2021–2022 games
Why encrypt?	Prevents easy recovery of classes, methods, fields and strings by reverse-engineering tools
Why search works	Unity's IL2CPP runtime must decrypt the file into the standard format in memory, otherwise the game crashes
Root cause	The behavior is dictated by Unity's engine design, not by any individual game's protection scheme
Best search	h AF 1B B1 FA 1F 00 00 00 (full 8 bytes) after the game has fully loaded

